



IA CORPORATIVA COM JAVA E LANGCHAIN4J

AULA 10 - CONECTANDO TOOLS E
GERENCIANDO MEMÓRIA



OBJETIVOS DA AULA

- ❑ Conectar a classe de Tools ao AiService para dar ao agente acesso às suas capacidades de negócio.
- ❑ Discutir a importância crítica da ChatMemory para agentes, funcionando também como um log de transações de estado.
- ❑ Implementar um ChatMemoryProvider para gerenciar o ciclo de vida e o escopo da memória de cada conversa.
- ❑ Habilitar o log detalhado das interações com o LLM para observar o ciclo ReAct (Reason + Act) em tempo real.
- ❑ Executar e testar o agente de ponta a ponta, analisando os logs para visualizar a chamada de Tool e a formulação da resposta.



CONECTANDO AS TOOLS AO AGENTE

CONECTANDO AS TOOLS AO AGENTE

- ❑ Na aula anterior, criamos a BookingTools, mas nosso AiService ainda não sabe de sua existência. Precisamos registrar explicitamente essa classe de Tools para que o Langchain4j possa construir o "manifesto de Tools" e enviá-lo ao LLM.
- ❑ A Anotação @RegisterAiService: Esta anotação possui um atributo tools que aceita um array de classes que contêm métodos anotados com @Tool.
- ❑ Passo 1: Atualizar a Interface PackageExpert
 - ❑ Modifique a interface PackageExpert para registrar a BookingTools.

CONECTANDO AS TOOLS AO AGENTE

```
package dev.ia;

import dev.langchain4j.service.MemoryId;
import dev.langchain4j.service.SystemMessage;
import dev.langchain4j.service.UserMessage;
import io.quarkiverse.langchain4j.RegisterAiService;

// A anotação agora inclui a referência à nossa classe de Tools
@RegisterAiService(tools = BookingTools.class)

public interface PackageExpert {

    @SystemMessage("""
        Você é um assistente virtual da 'Mundo Viagens', um especialista em nossos pacotes de viagem e reservas.
        Sua principal responsabilidade é responder às perguntas dos clientes de forma amigável e precisa,
        baseando-se exclusivamente nas informações contidas nos documentos que lhe foram fornecidos (RAG)
        ou utilizando as ferramentas disponíveis para interagir com o sistema de reservas.
        Nunca invente informações ou use conhecimento externo.
        Se a resposta para uma pergunta não estiver nos documentos e nenhuma ferramenta puder ajudar,
        você deve responder educadamente:
        'Desculpe, mas não tenho informações sobre isso. Posso ajudar com mais alguma dúvida sobre nossos pacotes?'
        """)

    String chat(@MemoryId String memoryId, @UserMessage String userMessage);
}
```

A IMPORTÂNCIA DA MEMÓRIA PARA AGENTES

A IMPORTÂNCIA DA MEMÓRIA PARA AGENTES

- ❑ Para um chatbot de Q&A, a memória serve para lembrar o que foi dito.
 - ❑ Para um agente, a memória é também log de transações de estado do ciclo ReAct.
- ❑ Ela armazena não apenas as mensagens do usuário e do assistente, mas também:
 - ❑ A requisição de execução da Tool gerada pelo LLM.
 - ❑ O resultado da execução da Tool retornado pelo nosso código Java.
- ❑ Sem esse histórico completo, o agente não conseguiria executar tarefas em várias etapas, pois o LLM não teria a "Observação" do resultado da ação anterior para basear seu próximo "Pensamento".
- ❑ Gerenciamento de Memória por Conversa:
 - ❑ Em um sistema real, múltiplos usuários interagem simultaneamente.
 - ❑ Precisamos garantir que a memória de cada conversa seja isolada.
 - ❑ O Langchain4j gerencia isso através do ChatMemoryProvider e da anotação @MemoryId.

CONFIGURANDO UM CHATMEMORYPROVIDER



CONFIGURANDO UM CHATMEMORYPROVIDER

- ❑ Controle Explícito sobre a Memória:
 - ❑ Embora o Quarkus forneça uma memória padrão, para agentes é uma boa prática definir explicitamente nosso provedor de memória para garantir o comportamento desejado.
- ❑ MessageWindowChatMemory:
 - ❑ É uma implementação que mantém um número fixo das últimas mensagens, descartando as mais antigas. É uma estratégia simples e eficaz para evitar que o contexto cresça indefinidamente.
- ❑ Passo 1: Criar o Provedor de Memória
 - ❑ Vamos criar um bean CDI que fornece instâncias de ChatMemory para cada sessão.

CONFIGURANDO UM CHATMEMORYPROVIDER

```
package dev.ia;

import dev.langchain4j.memory.ChatMemory;
import dev.langchain4j.memory.chat.MessageWindowChatMemory;
import dev.langchain4j.store.memory.chat.InMemoryChatMemoryStore;
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.enterprise.inject.Produces;

@ApplicationScoped
public class ChatMemoryConfig {

    // Produz um bean de ChatMemory para cada nova sessão de chat.
    @Produces
    public ChatMemory chatMemory() {
        return MessageWindowChatMemory.builder()
            .maxMessages(20) // Mantém as últimas 20 mensagens na memória
            .chatMemoryStore(new InMemoryChatMemoryStore())
            .build();
    }
}
```

CONFIGURANDO UM CHATMEMORYPROVIDER

- ❑ Análise Técnica:
 - ❑ @Produces:
 - ❑ Este método é um produtor CDI. O Quarkus o invocará para criar um bean ChatMemory.
 - ❑ MessageWindowChatMemory:
 - ❑ Estamos configurando cada memória para reter as últimas 20 mensagens.
 - ❑ InMemoryChatMemoryStore:
 - ❑ Por padrão, a memória de cada sessão é armazenada no heap. Para produção, isso poderia ser substituído por um RedisChatMemoryStore ou um JdbcChatMemoryStore para persistir as conversas.

HABILITANDO A "OBSERVABILIDADE" DO AGENTE

HABILITANDO A "OBSERVABILIDADE" DO AGENTE

- ❑ Vendo o Agente "Pensar":
 - ❑ Para depurar e entender o ciclo ReAct, é interessante ver as requisições e respostas trocadas com o LLM.
 - ❑ O Langchain4j permite habilitar isso com duas propriedades simples.
- ❑ Passo 1: Atualizar o application.properties
 - ❑ Adicione as seguintes linhas para habilitar o log completo:

```
# --- Configuração de Logging do LangChain4j ---
```

```
# Loga a requisição completa enviada para o LLM (incluindo o manifesto de Tools)
```

```
quarkus.langchain4j.log-requests=true
```

```
# Loga a resposta completa recebida do LLM (incluindo a chamada de Tool)
```

```
quarkus.langchain4j.log-responses=true
```

- ❑ Com essas configurações, cada interação com o agente irá imprimir no console o JSON exato enviado e recebido da API do Ollama, nos dando visibilidade total sobre o ciclo de raciocínio.

TESTANDO O AGENTE DE PONTA A PONTA

TESTANDO O AGENTE DE PONTA A PONTA

- ❑ A Hora da Verdade: Agora vamos fazer uma solicitação que exige que o agente use a suas novas Tools.
- ❑ Passo 1: Iniciar a Aplicação
 - ❑ `mvn quarkus:dev`
- ❑ Passo 2: Fazer uma Solicitação de Ação
 - ❑ Use o curl para pedir o cancelamento de uma reserva. Note que a frase é em linguagem natural, não uma chamada de API.

```
curl -X POST -H "Content-Type: text/plain" \  
-d "Por favor, cancele minha reserva 67890. Meu último nome é Smith." \  
http://localhost:8080/travel
```

ANALISANDO OS LOGS

- ❑ Observando o Ciclo ReAct no Console:
 - ❑ Ao executar o comando curl, você verá uma série de logs. Vamos analisá-los passo a passo:
- ❑ Request para o LLM (Pensamento): Você verá um grande JSON sendo enviado para o Ollama. Ele conterá:
 - ❑ A SystemMessage que definimos.
 - ❑ A UserMessage: "Por favor, cancele minha reserva 67890. Meu último nome é Smith."
 - ❑ A seção tools, com o manifesto JSON gerado a partir da nossa classe BookingTools.
- ❑ Response do LLM (Ação - Tool Call):
 - ❑ A resposta do Ollama não será um texto, mas um JSON pedindo para executar uma Tool:

```
{  
  "tool_calls": [{...}]  
}
```

ANALISANDO OS LOGS

- ❑ Execução da Tool (Ação - Código Java):
 - ❑ O Langchain4j intercepta essa resposta, invoca nosso método `BookingTools.cancelBooking(67890, "Smith")` e captura o retorno (String).
- ❑ Novo Request para o LLM (Observação -> Pensamento):
 - ❑ O Langchain4j inicia um novo ciclo, enviando todo o histórico (incluindo a chamada da Tool e seu resultado) de volta para o LLM.
- ❑ Response Final do LLM (Ação - Resposta Final):
 - ❑ O LLM agora "vê" que a Tool foi executada com sucesso e gera a resposta final em linguagem natural que é retornada ao usuário, algo como: "Sua reserva 67890 foi cancelada com sucesso. O status atual é: CANCELLED."

RESUMO E PRÓXIMOS PASSOS

RESUMO E PRÓXIMOS PASSOS

- ❑ O que Construimos Hoje?
 - ❑ Conectamos nossa classe BookingTools ao AiService, tornando as capacidades de negócio disponíveis para o agente.
 - ❑ Implementamos um ChatMemoryProvider para garantir que cada sessão de chat tenha uma memória isolada e com tamanho definido.
 - ❑ Configuramos o logging para obter visibilidade total do ciclo de raciocínio do agente.
 - ❑ Executamos um teste de ponta a ponta e analisamos os logs para ver, na prática, como o agente traduz uma intenção em linguagem natural em uma chamada de método Java.
- ❑ Próxima Aula:
 - ❑ Nosso agente já consegue usar uma Tool para completar uma tarefa simples.
 - ❑ Na sequência, vamos explorar cenários mais complexos de tarefas de várias etapas, onde o agente precisa coletar informações ao longo de várias iterações antes de poder executar uma ação final.

ATÉ A PRÓXIMA!